Natural Language Processing — Day 2

Transformers, Pretrained Models & Applied NLP

1. Transformer Recap

Day 1 covered classical text representation (BoW, TF-IDF, Word2Vec). Everything in Day 2 builds on the **Transformer architecture** — specifically **Attention** and **Self-Attention** (covered in Deep Learning, Module 5).

```
Attention(Q, K, V) = softmax( Q . K^T / sqrt(d_k) ) . V

# Self-Attention: Q, K, V all derived from the same sequence
# -> every token can directly relate to every other token, regardless of distance
```

Because Transformers have no recurrence, they process an entire sequence in parallel — this is what makes today's large-scale pretrained models computationally feasible.

2. Tokenization for Transformers

Classical tokenization (splitting on whitespace/words) breaks down on rare or unseen words (out-of-vocabulary / OOV problem). Transformer models instead use **subword tokenization**, which breaks words into smaller reusable pieces.

Method	Idea	Used by
BPE (Byte Pair Encoding)	Iteratively merges the most frequent adjacent character/subword pairs	GPT-2, GPT-3, RoBERTa
WordPiece	Similar to BPE, but merges based on likelihood gain rather than raw frequency	BERT
SentencePiece	Treats text as a raw stream (no whitespace assumption); language-agnostic	T5, XLNet, ALBERT

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer.tokenize("unbelievable transformation")
print(tokens)
# ['unbelievable', 'transformation']  -- or subword pieces for rarer words, e.g.:
# ['un', '##believable', 'transformation']
```

This subword approach means the model can represent virtually **any** word — even ones never seen during training — by composing it from known subword pieces.

3. Pretrained Language Models

The core shift in modern NLP: instead of training a model from scratch for every task, **pretrain once** on massive amounts of text, then **adapt** to specific tasks.

BERT (Bidirectional Encoder Representations from Transformers)

Reads text **bidirectionally** — using context from both the left and right of a word simultaneously. Pretrained using two objectives:

Objective	What it does
Masked Language Modeling (MLM)	Randomly masks ~15% of tokens and trains the model to predict them from surrounding context
Next Sentence Prediction (NSP)	Trains the model to predict whether sentence B actually follows sentence A

```
from transformers import pipeline
fill_mask = pipeline("fill-mask", model="bert-base-uncased")
print(fill_mask("Paris is the [MASK] of France."))
# top prediction: "capital"
```

GPT Family

Uses **autoregressive / causal language modeling** — predicts the next token using only the tokens that came before it (left-to-right), never looking ahead. This makes GPT naturally suited for text generation.

```
next_token_probability = P( token_t | token_1, token_2, ..., token_(t-1) )
# Trained purely by predicting "what comes next", one token at a time
```

T5 / BART — Encoder-Decoder Models

Frame **every** NLP task as text-to-text: translation, summarization, classification, and question answering all become "input text in, output text out" — using the full Encoder-Decoder Transformer architecture rather than encoder-only (BERT) or decoder-only (GPT).

```
# T5 style framing:
"translate English to French: The house is small"   -> "La maison est petite"
"summarize: <long article text>"                    -> "<short summary>"
"cola sentence: The dog run fast"                    -> "unacceptable"
```

Transfer Learning in NLP: Pretraining vs Fine-Tuning

Stage	What happens
Pretraining	Model learns general language patterns from huge, unlabeled text corpora (expensive, done once)
Fine-tuning	Pretrained model is further trained on a small, labeled, task-specific dataset (cheap, done per-task)

```
from transformers import AutoModelForSequenceClassification, Trainer

model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
# ... fine-tune "model" on your own labeled dataset with Trainer/TrainingArguments ...
```

Sentence-BERT (SBERT)

Produces a single dense embedding for an **entire sentence** (rather than per-token embeddings), specifically optimized so that **cosine similarity** between sentence embeddings reflects semantic similarity — makes large-scale semantic search practical and fast.

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
emb1 = model.encode("How do I reset my password?")
emb2 = model.encode("I forgot my password, what should I do?")
print(util.cos_sim(emb1, emb2))   # high similarity score
```

4. Applied NLP Tasks

Machine Translation

Translating text from a source language to a target language — typically an encoder-decoder Transformer model.

Text Summarization

Type	Approach
Extractive	Selects and stitches together key existing sentences from the source text
Abstractive	Generates new sentences that capture the meaning, potentially using words not in the source

```
from transformers import pipeline
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
summarizer(long_article_text, max_length=60, min_length=20)
```

Question Answering

Type	Approach
Extractive QA	Finds and returns the exact answer span within a given context passage
Generative QA	Composes a new answer in natural language, not necessarily copied verbatim from the text

```
qa = pipeline("question-answering")
qa(question="Who wrote the play?", context="Hamlet was written by William Shakespeare.")
# {'answer': 'William Shakespeare', ...}
```

Text Generation

Producing new, coherent text given a prompt — the core capability behind GPT-style models and chatbots.

Semantic Search / Text Similarity

Finding the most relevant documents/passages for a query by comparing **embeddings** rather than exact keyword matches — this embedding-based retrieval is the exact mechanism that powers **RAG (Retrieval-Augmented Generation)**.

Chatbots & Dialogue Systems

Combine language understanding, context/memory across turns, and text generation to hold multi-turn conversations.

5. Evaluation Metrics

Metric	Used for	What it measures
Perplexity	Language Models	How "surprised" the model is by real text (lower = better)
BLEU	Translation	N-gram overlap between generated and reference text (precision-focused)
ROUGE	Summarization	N-gram/sequence overlap (recall-focused)
METEOR	Translation/Summarization	Overlap with synonym & stemming matching, more human-aligned than BLEU
Accuracy/Precision/Recall/F1	Classification	Standard classification performance metrics
Exact Match & F1	Question Answering	Whether predicted answer span exactly matches (EM) or overlaps (F1) the ground t

```
Perplexity = 2 ^ ( -(1/N) * sum( log2( P(word_i | context) ) ) )
```

6. Tools & Libraries

Library	Used for
NLTK, spaCy	Classical NLP — tokenization, POS tagging, NER, parsing
Gensim	Word2Vec, LDA topic modeling
Hugging Face Transformers	Pretrained BERT/GPT/T5 models and ready-to-use pipelines
Sentence-Transformers	Sentence embeddings for semantic search (SBERT)

7. Bridge to LLMs & RAG

Everything in this module sets up the next stage of the journey — Large Language Models and Retrieval-Augmented Generation.

- **Prompt Engineering basics** — how the exact wording/structure of an instruction shapes an LLM's output
- **Embeddings for retrieval** — the same SBERT-style embeddings used for semantic search become the retrieval backbone of RAG systems
- **Context windows & tokenization limits** — every model has a maximum number of tokens it can process at once, directly shaped by the tokenizer covered in Section 2
- **Fine-tuning vs Prompting vs RAG** — three different strategies for adapting a general pretrained model to a specific problem, each with different cost/data/flexibility tradeoffs

Strategy	When to use it
Prompting	Fast, no training needed; works when the model already "knows" enough
Fine-tuning	When you need the model to reliably follow a specific style/format/domain and have labeled data
RAG	When answers must be grounded in specific, up-to-date, or private documents/knowledge

Key Takeaways

- Modern NLP tokenizes text into **subwords** (BPE, WordPiece, SentencePiece) to eliminate the out-of-vocabulary problem entirely.
- **BERT** is bidirectional and answers "what fits here, given everything around it?" — **GPT** is autoregressive and answers "what comes next, given everything so far?"
- **T5/BART** unify every NLP task into a single text-to-text framing using a full encoder-decoder Transformer.
- The core paradigm shift in NLP: **pretrain once** on massive text, then **fine-tune** cheaply per task — rather than training from scratch every time.
- **SBERT** turns whole sentences into embeddings optimized for similarity search — the exact mechanism that underlies semantic search and RAG retrieval.
- Evaluation metrics are task-specific: **Perplexity** for language models, **BLEU/ROUGE/METEOR** for generation tasks, **Accuracy/F1** for classification, **Exact Match/F1** for QA.
- This module directly bridges into **LLMs and RAG** — prompting, fine-tuning, and retrieval-augmented generation are three distinct ways to adapt a pretrained model to a real problem.

Module 6 (NLP) Complete — Next: Large Language Models (LLMs) and RAG (Retrieval-Augmented Generation).